High-Performance Concurrent Bidding System for Banks

Welcome to the **NexBid** project architecture and documentation. This project provides the foundation for an enterprise-grade, high-concurrency auction and bidding system specifically tailored for inter-bank trading and institutional asset liquidation.

Executive Summary

In high-stakes financial environments, millisecond latency and transactional integrity are paramount. When multiple financial institutions bid on a single asset simultaneously, the underlying system must handle massive concurrent requests without data corruption, race conditions, or performance degradation.

NexBid addresses these challenges by leveraging **Spring Boot** for robust business logic, **Redis** for distributed locking and in-memory processing, and **WebSockets** for low-latency market data broadcasting.

Core Architecture & Technologies

- **Backend Framework:** Spring Boot 3.x (Java 21)
- **Database:** PostgreSQL (Relational persistence, ACID compliance for final trade settlements)
- **In-Memory Datastore / Cache:** Redis
- **Real-Time Communication:** WebSockets with STOMP protocol
- **Infrastructure:** Docker & Docker Compose

Backend

This system implements several advanced software engineering patterns to achieve high throughput and strict transactional reliability:

1. Distributed Locking (Redis & Redisson)

The Problem: If Bank A and Bank B bid on the same asset concurrently, traditional relational databases may suffer from race conditions or heavy lock contention. **The Solution:** The system utilizes Redis-based distributed locking (via Redisson). Upon receiving a bid, the application acquires a distributed lock on the specific `auction_id`. Subsequent bids are queued or rejected immediately if they fall below the new highest bid, ensuring thread safety across multiple server nodes.

2. In-Memory Bid Processing (CQRS Pattern)

Writing every discrete bid directly to a relational database synchronously creates a significant I/O bottleneck. To mitigate this, we implement a lightweight **Command Query Responsibility Segregation (CQRS)** pattern:

- **Writes:** Bids are validated and written to Redis instantly, providing extremely high throughput and sub-millisecond response times.
- **Asynchronous Persistence:** Background worker threads consume validated bids from Redis streams and batch-persist them to PostgreSQL asynchronously, removing the database from the critical path.

3. Real-Time WebSockets

Clients are not required to poll the server for state changes. Once a valid bid is processed in Redis, a message is published to a WebSocket message broker, which pushes the updated market data to all connected institutional clients in real-time.

4. Optimistic Locking (JPA)

As a secondary safeguard at the database persistence layer, JPA Optimistic Locking (`@Version`) is employed to guarantee that no dirty writes occur during the asynchronous batch-persistence process.

Key Features

- **Live Auction Dashboard:** Real-time visibility into active order books.
- **Sub-Millisecond Latency:** Redis caching ensures validation and processing occur entirely in-memory.
- **High Concurrency:** Designed to handle thousands of transactions per second on a single asset.
- **Audit Trail:** Comprehensive bid history is persisted to PostgreSQL for regulatory compliance and auditability.

Developed as a reference architecture for high-throughput distributed financial systems.

Roadmap

1. ~~**Phase 1: Project Scaffolding & Infrastructure**~~ (Completed)
    - Initialized Spring Boot, PostgreSQL, and Redis environments.
2. **Phase 2: Core Bidding Engine** (In Progress)
    - Implementation of Redis distributed locking and strict bid validation logic.
3. **Phase 3: Real-Time Broadcasting**
    - Integration of WebSockets for live market data updates.
4. **Phase 4: Async Persistence**
    - Implementation of background workers for reliable Redis-to-PostgreSQL data replication.
5. **Phase 5: API & UI Integration**
    - Development of RESTful APIs and a demonstration frontend.